Name:Khushi Singh
Class:D20C Roll no.34

Experiment No.6

Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE

Aim: Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.

Course Outcomes (CO): CO3, CO5, CO6

Theory

1. Relevance of Required Statements in Solidity Programs

The require statement works as a guard condition placed at the start of a function. It checks that a condition holds before the rest of the function is allowed to run — if the condition evaluates to false, execution stops immediately and every state change made so far in that transaction is rolled back to how it was before the call began. This all-or-nothing behaviour prevents an invalid or unauthorized call from leaving the contract's storage in an inconsistent state.

In the Ballot contract used in this experiment, require is used in exactly this protective role in three places:

- In giveRightToVote, to confirm that only the chairperson can grant voting rights, and that the target address has not already voted or already been given a weight.
- In delegate, to confirm the caller actually has the right to vote, has not already voted, is not delegating to themselves, and is not delegating into a loop.
- In vote, to confirm the caller has a non-zero weight and has not already voted.

Each require also carries a short string message, which is returned to the caller if the check fails — this is what shows up as the revert reason in Remix's console (for example, "Only chairperson can give right to vote."), which makes debugging and testing far easier than a plain revert.

2. The Keywords mapping, storage and memory

mapping — A mapping is Solidity's key-value data structure, declared as mapping(keyType => valueType). The contract declares mapping(address => Voter) public voters;, which links every Ethereum address to its own Voter struct. This is what lets the contract look up a specific account's voting weight, voted status and delegate in constant time. Mappings have no length and cannot be iterated over directly, which keeps lookups cheap but means there is no built-in way to list every voter.

storage — storage is the contract's permanent, on-chain state. State variables such as chairperson, voters and proposals live in storage by default, and any change made to them persists across transactions and calls. Writing to storage is the most gas-expensive operation in the EVM, which is why the contract only writes to it when a value genuinely needs to survive (e.g. sender.voted = true; inside vote).

memory — memory is temporary and exists only for the duration of a single function call; anything held there is discarded once the function returns. It is used for function arguments, local variables and intermediate values that do not need to be kept on-chain — for instance, the proposalNames array passed into the constructor, or the string memory return value of winnerName(). Because memory is far cheaper than storage, using it correctly for anything that doesn't need to persist is an important gas-optimisation habit when writing Solidity.

3. bytes32 versus string for Proposal Names

The original version of the Ballot contract (as provided in the reference material) stores each proposal's name as bytes32 rather than string. bytes32 is a fixed-size, 32-byte type: because its size never changes, it is simple and cheap for the EVM to store and compare, but it hard-caps every proposal name to 32 bytes and stores raw byte data rather than human-readable text — which is why tools like a bytes32-to-string converter are needed just to read the values back.

string, in contrast, is a dynamically-sized type that can hold text of any length and is directly readable without any conversion step. It is far friendlier for both the developer and the end user, at the cost of extra gas: dynamic types need extra bookkeeping (length, offset) inside the EVM's storage and ABI encoding, which makes them somewhat more expensive to store and compare than a fixed-size bytes32.

For this experiment, the contract was modified so that the Proposal struct's name field, the constructor's proposalNames parameter, and the return type of winnerName() all use string directly, instead of bytes32. This removed the need for any bytes32-to-string conversion step when deploying with human-readable names such as "Sandy", "Ruturaj" and "Sumaant", at the small extra gas cost that comes with using a dynamic type.

Code

The following listing is the modified VotingBallot.sol used for this experiment, with the Proposal.name field, the constructor parameter and the return type of winnerName() changed from bytes32 to string:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {
    struct Voter {
        uint weight;
        bool voted;
        address delegate;
        uint vote;
    }

    struct Proposal {
        string name;      // changed from bytes32 to string
        uint voteCount;
    }

    address public chairperson;

    mapping(address => Voter) public voters;
```

```
Proposal[] public proposals;

constructor(string[] memory proposalNames) {
    chairperson = msg.sender;
    voters[chairperson].weight = 1;

    for (uint i = 0; i < proposalNames.length; i++) {
        proposals.push(Proposal({
            name: proposalNames[i],
            voteCount: 0
        }));
    }
}

function giveRightToVote(address voter) external {
    require(
        msg.sender == chairperson,
        "Only chairperson can give right to vote."
    );
    require(
        !voters[voter].voted,
        "The voter already voted."
    );
    require(voters[voter].weight == 0, "Voter already has the right to vote.");
    voters[voter].weight = 1;
}

function delegate(address to) external {
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted.");
    require(to != msg.sender, "Self-delegation is disallowed.");

    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;
        require(to != msg.sender, "Found loop in delegation.");
    }

    Voter storage delegate_ = voters[to];
    require(delegate_.weight >= 1);

    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {
        proposals[delegate_.vote].voteCount += sender.weight;
    } else {
        delegate_.weight += sender.weight;
    }
```

```
    }

    function vote(uint proposal) external {
        Voter storage sender = voters[msg.sender];
        require(sender.weight != 0, "Has no right to vote");
        require(!sender.voted, "Already voted.");
        sender.voted = true;
        sender.vote = proposal;

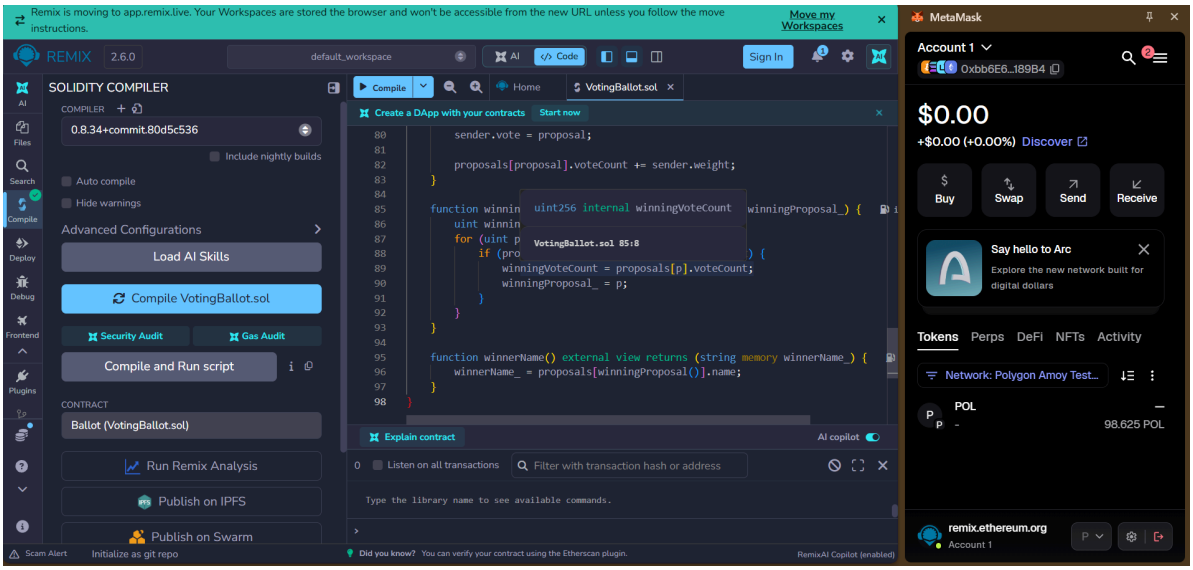
        proposals[proposal].voteCount += sender.weight;
    }

    function winningProposal() public view returns (uint winningProposal_) {
        uint winningVoteCount = 0;
        for (uint p = 0; p < proposals.length; p++) {
            if (proposals[p].voteCount > winningVoteCount) {
                winningVoteCount = proposals[p].voteCount;
                winningProposal_ = p;
            }
        }
    }

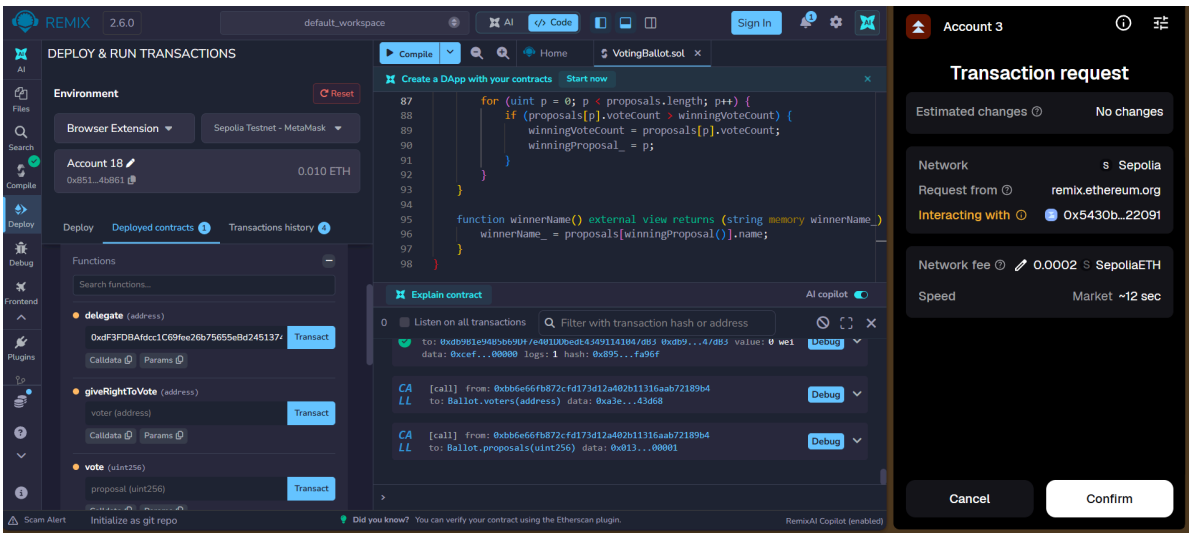
    function winnerName() external view returns (string memory winnerName_) {
        winnerName_ = proposals[winningProposal()].name;
    }
}
```

Output

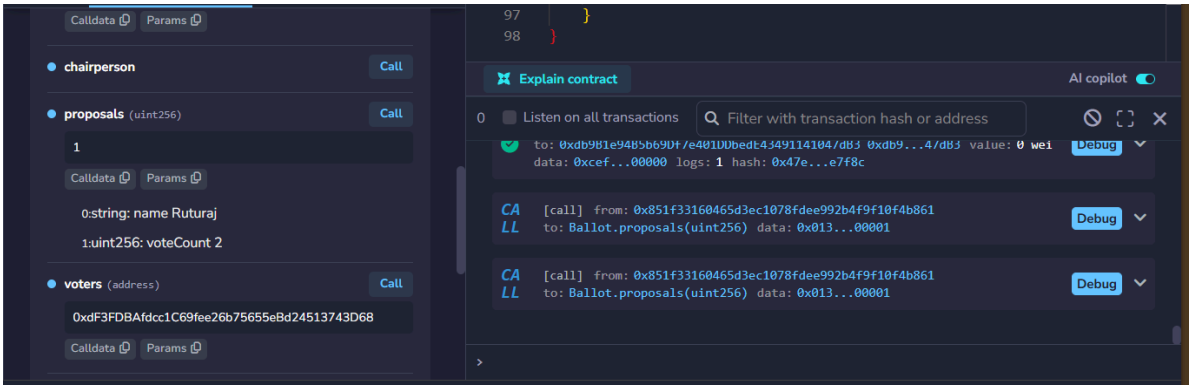
The contract was compiled and deployed from Remix IDE using the Injected Provider (MetaMask) environment on the Sepolia / Polygon Amoy testnets. The screenshots below trace the full flow: compiling the contract, deploying it with the proposal names ["Sandy", "Ruturaj", "Sumaant"], granting voting rights, delegating a vote, casting votes, and finally reading back the winning proposal.



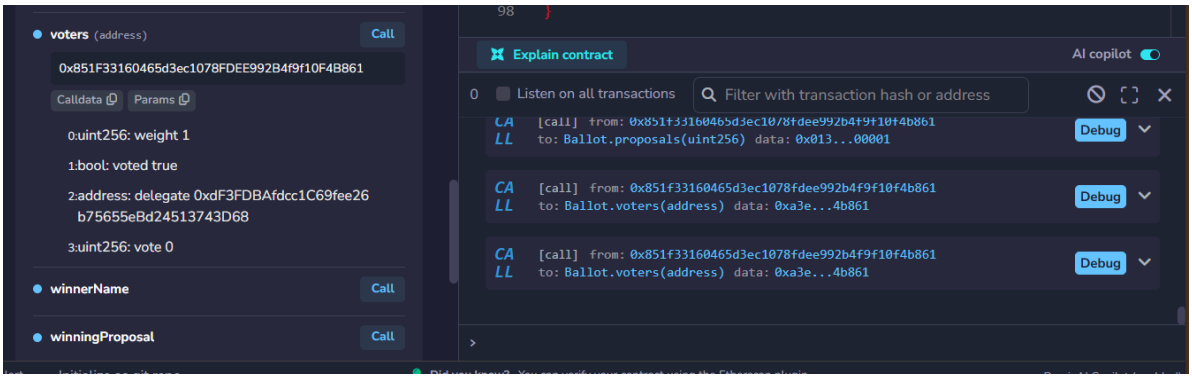
Solidity Compiler panel in Remix — VotingBallot.sol compiled successfully (compiler 0.8.34) with the MetaMask wallet connected on Polygon Amoy Testnet.



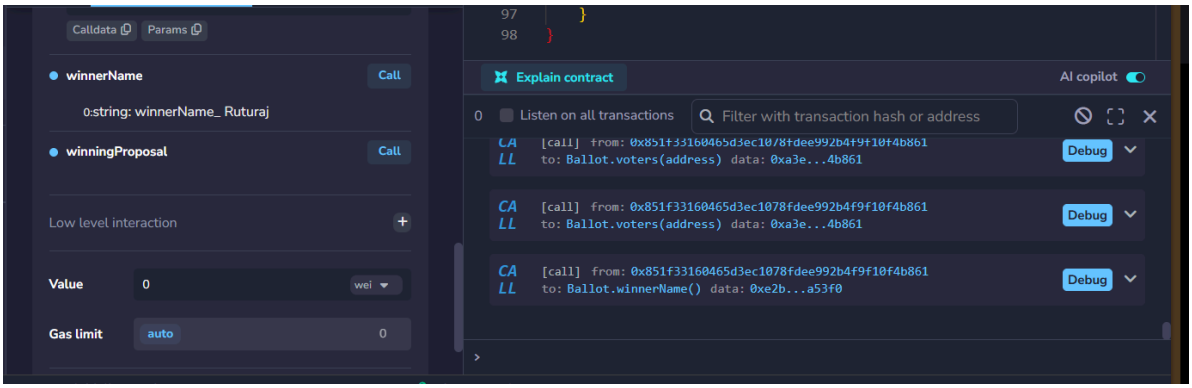
MetaMask transaction request generated while interacting with the deployed Ballot contract.



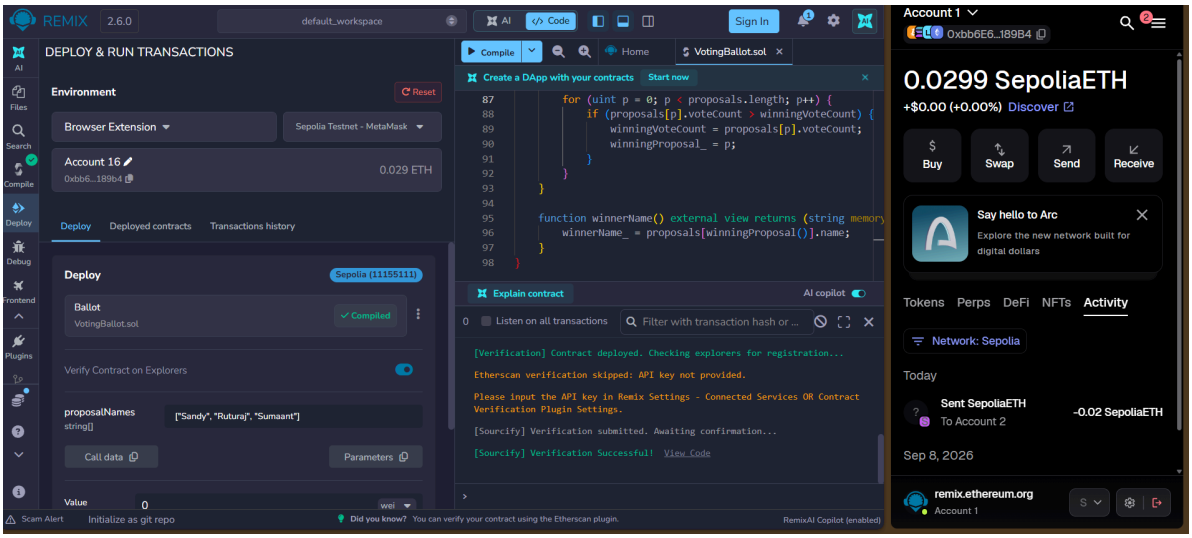
Reading state after deployment — chairperson, proposals(1) and voters(address) called from the deployed-contract panel.



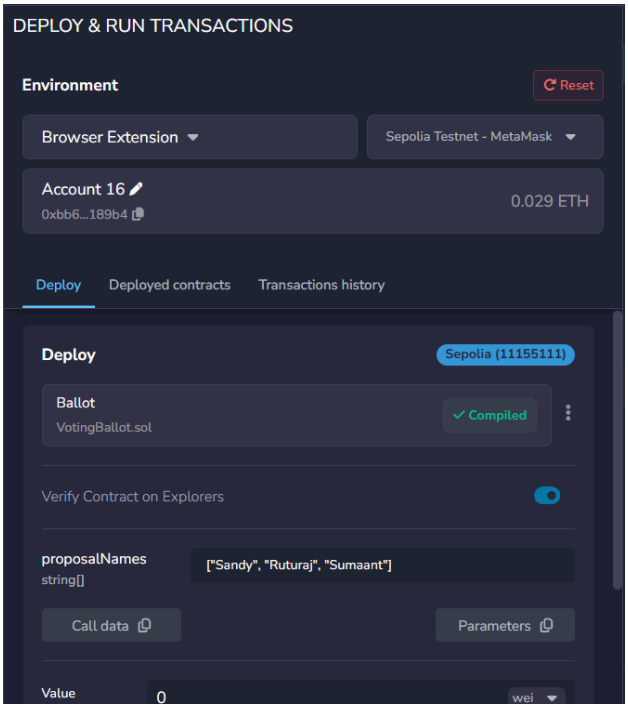
Delegation — the voters mapping for the delegating account, showing weight 1, voted = true, delegate set to another voter's address, and vote = 0.



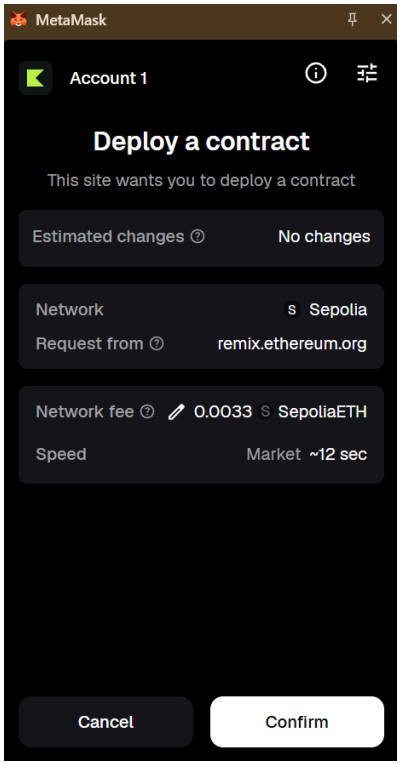
Reading the outcome functions winnerName and winningProposal after a delegated vote.



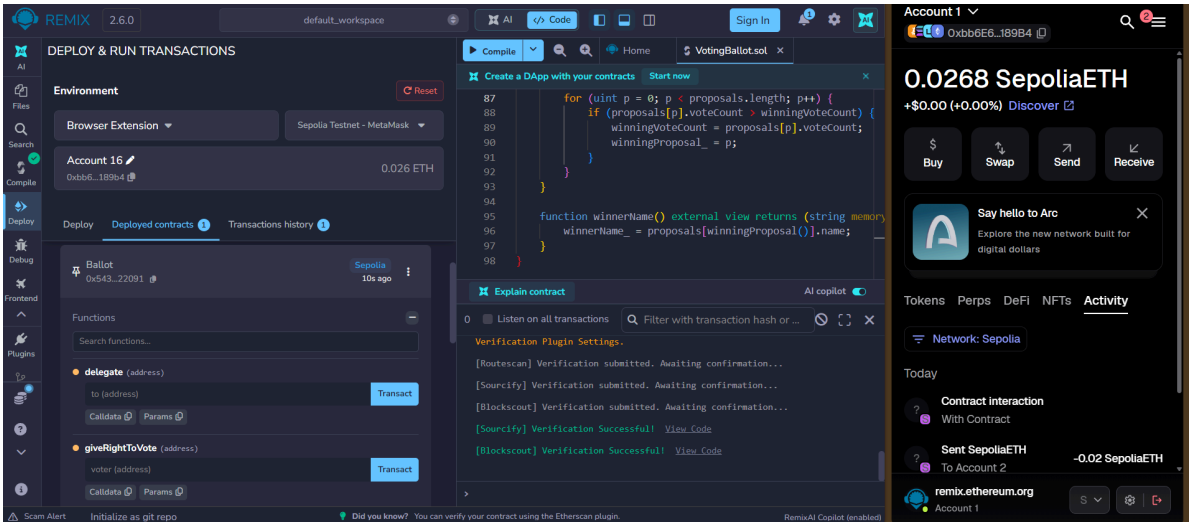
Deploying with "Verify Contract on Explorers" enabled on Sepolia, alongside the MetaMask activity feed showing the SepoliaETH balance and recent transfer.



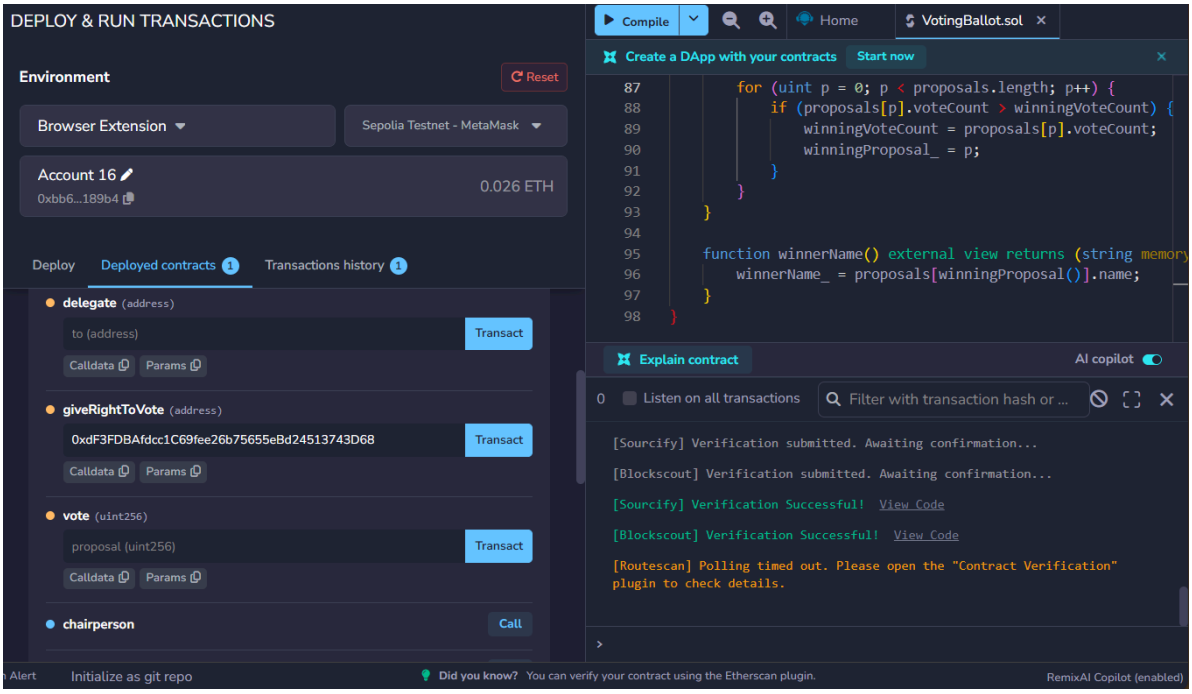
Deploy & Run Transactions panel with the constructor argument proposalNames set to ["Sandy", "Ruturaj", "Sumaant"].



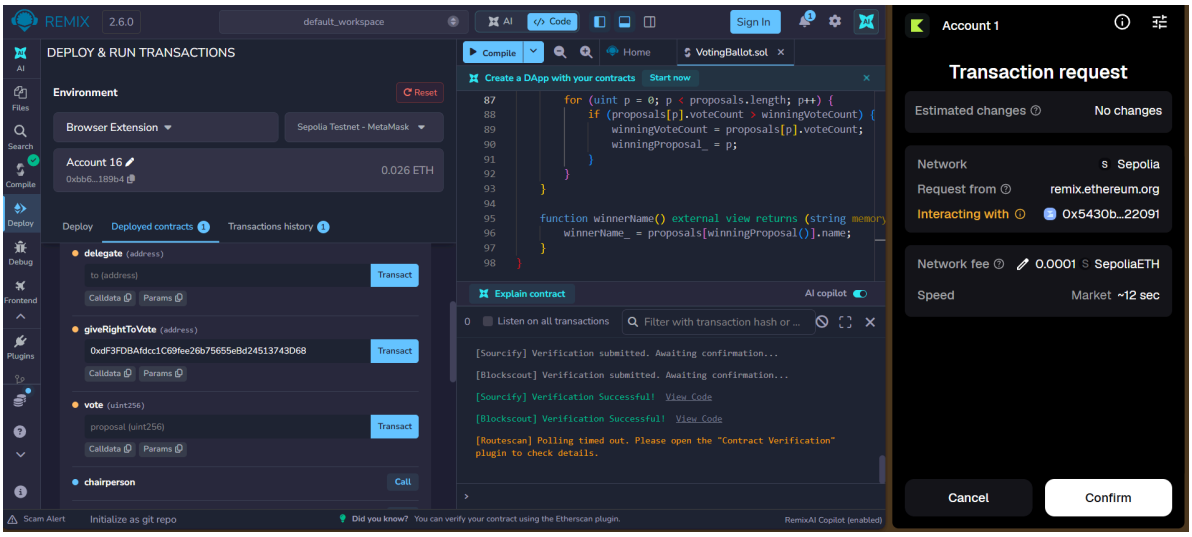
MetaMask prompt asking the account to confirm the contract-deployment transaction on the Sepolia testnet.



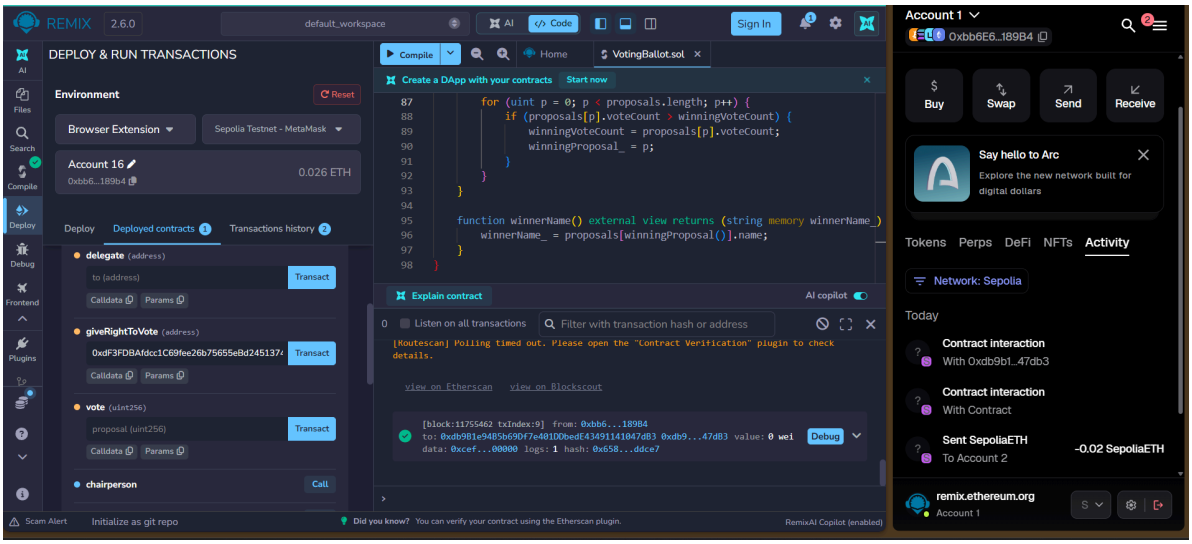
Deployed contract's functions (delegate, giveRightToVote, vote, chairperson) alongside the Sourcify/Blockscout verification log.



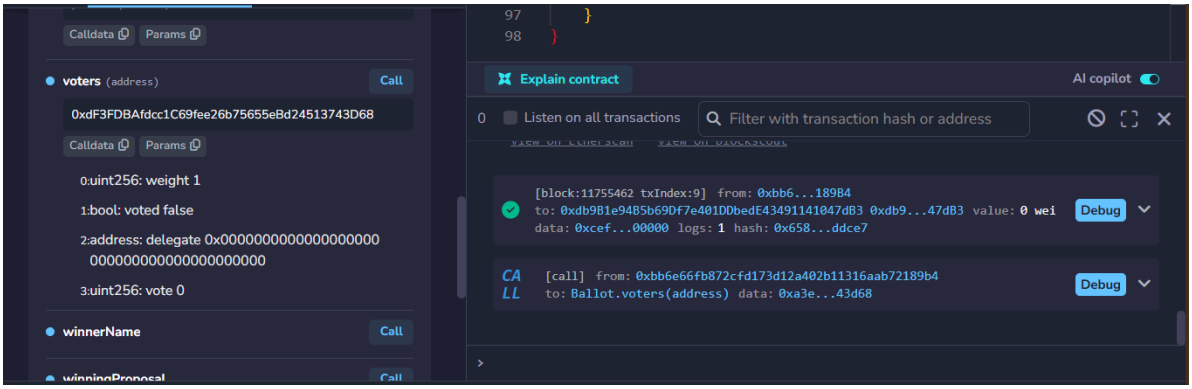
Deploy & Run Transactions panel after deployment, showing the delegate, giveRightToVote, vote and chairperson functions ready to be called, with verification messages in the console.



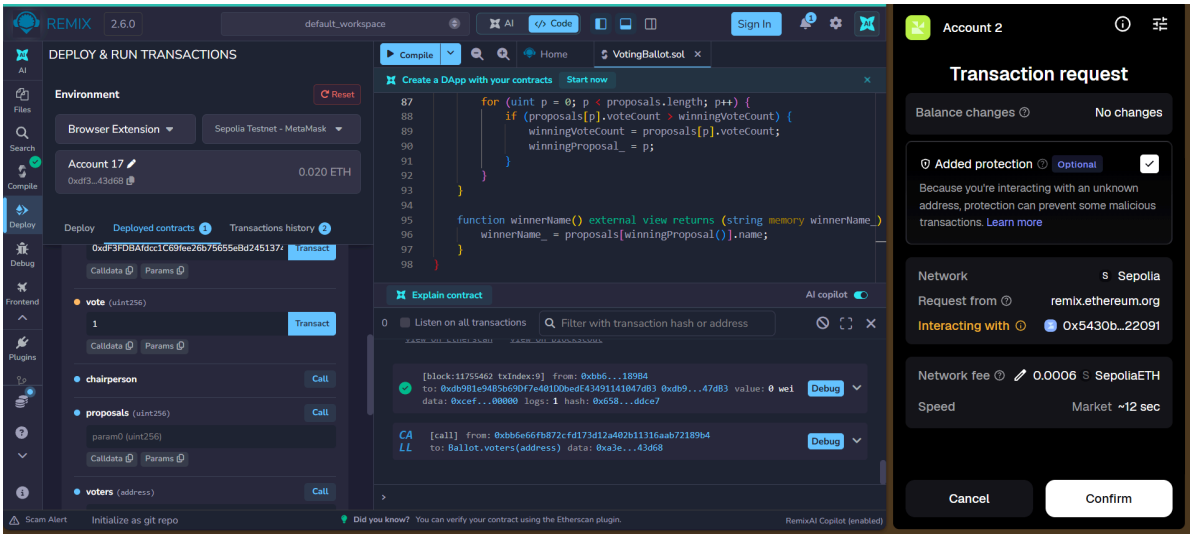
MetaMask transaction request confirming a call made from Account 1 to the deployed contract.



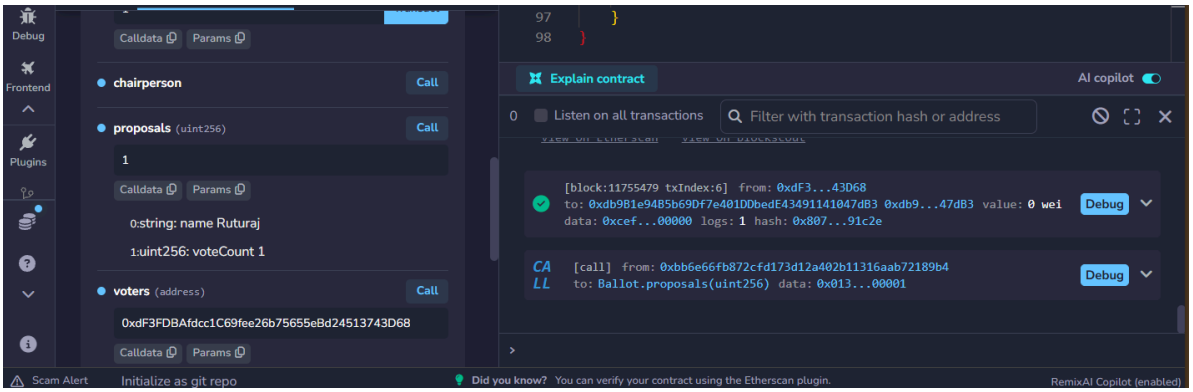
MetaMask activity log showing a contract interaction together with a SepoliaETH transfer to a second account.



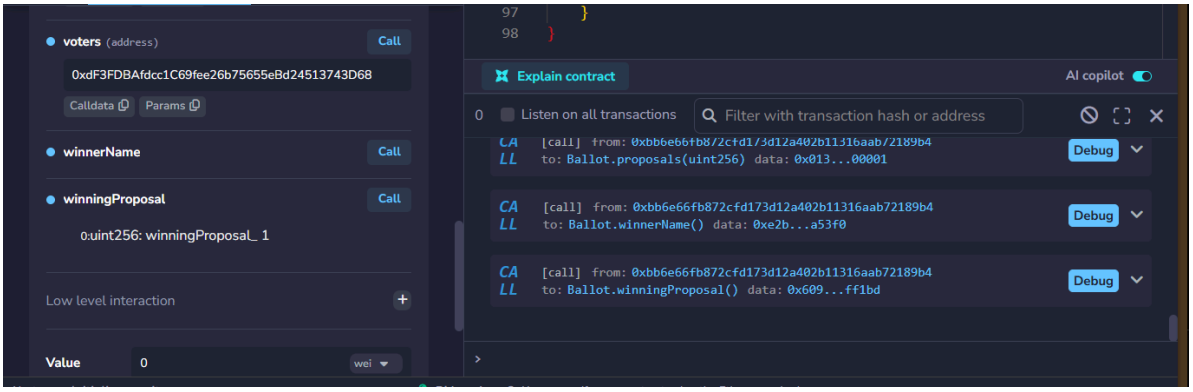
Result of calling voters(address) for a newly-registered voter — weight 1, voted = false, delegate unset, vote = 0.



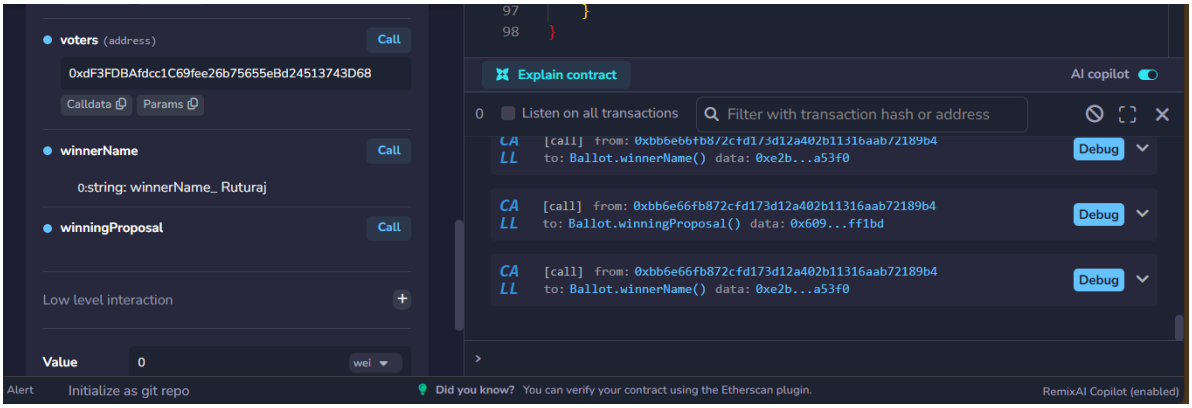
MetaMask transaction request raised from Account 2, with the wallet's "Added protection" warning shown for an unfamiliar address.



chairperson, proposals(1) and voters(address) calls after voting — proposal 1 ("Ruturaj") shows voteCount 1.



voters, winnerName and winningProposal calls — winningProposal_ resolves to index 1.



Final result of the experiment — winnerName_ returns "Ruturaj", confirming it as the winning proposal.

Conclusion

In this experiment, the Ballot voting smart contract was deployed and exercised end-to-end using Solidity and Remix IDE, connected through MetaMask to a live testnet. The require statement was studied as the primary mechanism for enforcing access control and input validation, with automatic state rollback on failure. The distinction between mapping, storage and memory was explored to understand how the contract organises persistent voter and proposal data versus temporary, function-scoped values. The Proposal.name field, the constructor and winnerName() were also modified from bytes32 to string, trading a small increase in gas cost for proposal names that are directly human-readable, removing the need for any bytes32-to-string conversion tooling. Deploying the contract, granting voting rights, delegating a vote, casting votes and reading back winnerName() and winningProposal() together confirmed that the modified contract behaves correctly on-chain, with "Ruturaj" resolving as the winning proposal.